

Rapport Final TER

PAC-MAN

Génération de labyrinthes
et Intelligence Artificielle

Réalisé par Nour Jazi
Nour Montasser

Encadré par Mr Gilles Menez

Groupe TER - S2 Groupe F

Année 2025-2026

Résumé

Ce projet TER consiste à développer un système complet autour du jeu Pac-Man combinant génération de labyrinthes, moteur de jeu web interactif, intelligence artificielle des fantômes et IA décisionnelle pour Pac-Man.

Le projet met particulièrement l'accent sur la comparaison d'algorithmes en temps réel : pathfinding (Greedy, BFS, A*) et recherche adversariale (Minimax, Alpha-Beta, Expectimax).

L'objectif était de construire une application pédagogique, interactive et extensible permettant d'observer concrètement différentes stratégies d'intelligence artificielle dans un environnement de jeu.

Table des matières

1	Introduction	2
2	Génération de labyrinthes	2
2.1	Principe général	2
2.2	Résultats	2
3	Architecture logicielle	3
3.1	Frontend	3
3.2	Backend	3
3.3	MongoDB	3
4	IA des fantômes	4
4.1	Architecture modulaire	4
4.2	Personnalités des fantômes	4
4.3	Modes des fantômes	4
5	Comparaison des algorithmes de pathfinding	4
5.1	Greedy	4
5.2	BFS	4
5.3	A*	4
6	Pac-Man Self-Play	5
6.1	Modélisation en arbre de jeu	5
6.2	Minimax	6
6.3	Alpha-Beta	6
6.4	Expectimax	6
6.5	Heuristiques et optimisations	6
7	Tests, statistiques et CI/CD	7
7.1	Statistiques temps réel	7
7.2	Tests automatisés	7
7.3	CI/CD	7
8	Conclusion et perspectives	8

1 Introduction

Pac-Man constitue un environnement particulièrement intéressant pour l'intelligence artificielle car il combine plusieurs problématiques importantes comme la navigation dans une grille, la prise de décision, la poursuite d'agents et l'optimisation en temps réel.

L'objectif de ce projet TER était de développer un système complet autour du jeu Pac-Man combinant génération procédurale de labyrinthes, moteur de jeu interactif et intelligence artificielle.

Le projet a été réalisé en trois grandes phases :

- **Phase 1** : génération automatique de labyrinthes et déploiement cloud de l'application.
- **Phase 2** : développement des intelligences artificielles des fantômes, incluant différents algorithmes de pathfinding et comportements.
- **Phase 3** : développement du Pac-Man Self-Play et des algorithmes de décision adversariale comme Minimax, Alpha-Beta et Expectimax.

Le système final permet de comparer plusieurs algorithmes de pathfinding et de recherche adversariale tout en observant leurs comportements et performances en temps réel à travers une interface web interactive.

2 Génération de labyrinthes

2.1 Principe général

Le système de génération repose sur une approche hybride combinant plusieurs techniques comme DFS (Depth First Search), Prim et Braid Maze.

L'objectif était de produire des labyrinthes proches de ceux de Pac-Man : fortement connectés, avec peu de culs-de-sac, contenant des cycles et adaptés au gameplay des fantômes.

La technique "Braid Maze" permet notamment de supprimer les dead ends afin de rendre les déplacements plus fluides.

Plusieurs éléments ont également été ajoutés automatiquement lors de la génération comme la Ghost House centrale, les tunnels wrap-around, les cycles supplémentaires ainsi que les pellets et power pellets afin de garantir un gameplay fluide et jouable.

2.2 Résultats

Les labyrinthes générés restent rapides même pour des tailles importantes.

Taille	Temps moyen	Qualité moyenne
15x15	~10 ms	89/100
30x30	~40 ms	87/100

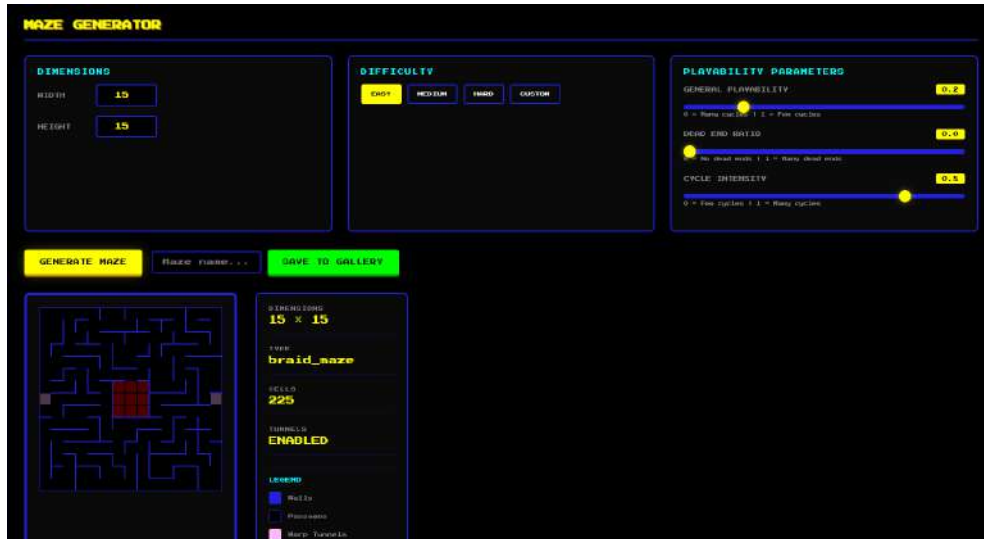


FIGURE 1 – Interface de génération de labyrinthes

3 Architecture logicielle

Le projet suit une architecture séparant frontend et backend.

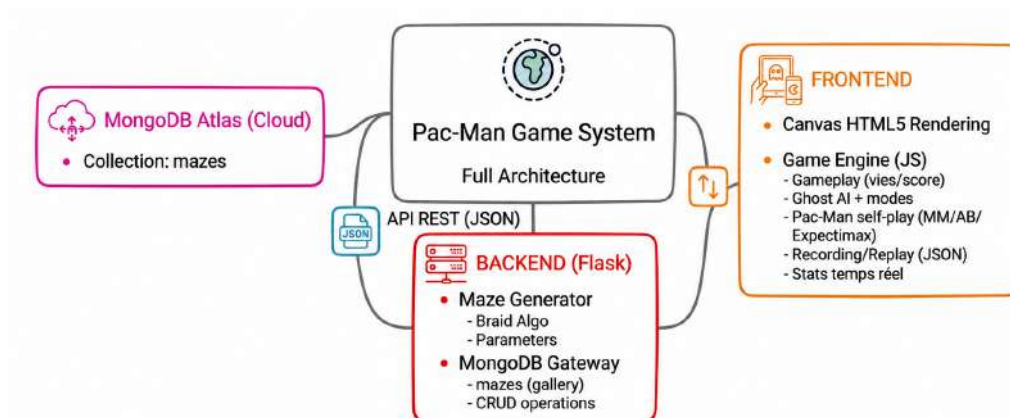


FIGURE 2 – Architecture logicielle du système

3.1 Frontend

Le frontend est développé en JavaScript avec Canvas HTML5. Il gère le moteur de jeu, l'affichage du labyrinthe, les animations, les statistiques temps réel ainsi que les interactions utilisateur.

3.2 Backend

Le backend repose sur Flask et assure la génération des labyrinthes, les appels API REST ainsi que les opérations de stockage et de gestion des données via MongoDB.

3.3 MongoDB

Une galerie MongoDB a été ajoutée afin de sauvegarder les labyrinthes générés, les recharger, les noter et conserver un historique des créations.

4 IA des fantômes

4.1 Architecture modulaire

L'IA des fantômes repose sur une séparation entre le ciblage, le pathfinding et le déplacement. Chaque fantôme possède sa propre personnalité mais partage les mêmes algorithmes de pathfinding, ce qui rend le système facilement extensible.

4.2 Personnalités des fantômes

Les quatre fantômes principaux de Pac-Man ont été reproduits :

- **Blinky** : poursuite directe de Pac-Man,
- **Pinky** : embuscade devant Pac-Man,
- **Inky** : comportement coopératif,
- **Clyde** : alternance poursuite/fuite.

4.3 Modes des fantômes

Afin de reproduire un comportement proche du jeu Pac-Man original, plusieurs modes de fonctionnement ont été implémentés pour les fantômes.

Le mode **Scatter** force temporairement les fantômes à rejoindre une zone spécifique du labyrinthe, tandis que le mode **Chase** active leur comportement de poursuite principal.

Lorsque Pac-Man mange un power pellet, les fantômes passent en mode **Frightened** et deviennent vulnérables avec un comportement plus aléatoire. Enfin, le mode **Eaten** permet au fantôme capturé de retourner automatiquement vers la Ghost House avant de réapparaître.

L'alternance entre ces états rend le gameplay plus dynamique et plus réaliste.

5 Comparaison des algorithmes de pathfinding

Trois algorithmes principaux ont été implémentés afin de comparer différents comportements de poursuite.

5.1 Greedy

Greedy choisit uniquement la direction localement la plus intéressante. Il est très rapide mais peut produire des chemins non optimaux.

5.2 BFS

Breadth First Search explore les nœuds niveau par niveau. Il garantit le plus court chemin mais explore davantage de nœuds.

5.3 A*

A* combine le coût réel du déplacement avec une heuristique Manhattan afin d'obtenir un excellent compromis entre performance et optimalité.

Algorithme	Optimal	Temps	Nœuds explorés
Greedy	Non	Très rapide	Très faible
BFS	Oui	Moyen	Élevé
A*	Oui	Rapide	Moyen



FIGURE 3 – Comparaison des algorithmes de pathfinding

6 Pac-Man Self-Play

La phase 3, en revanche comporte l'ajout d'une IA directement côté Pac-Man.

L'objectif était de permettre au jeu de prendre ses propres décisions en utilisant des algorithmes de recherche adversariale.

6.1 Modélisation en arbre de jeu

Le système modélise les futurs états du jeu sous forme d'un arbre de décisions. Pac-Man explore plusieurs coups à l'avance afin de choisir la meilleure stratégie.

Chaque état prend en compte la position des fantômes, les pellets restants, les possibilités de fuite ainsi que la progression dans le niveau.

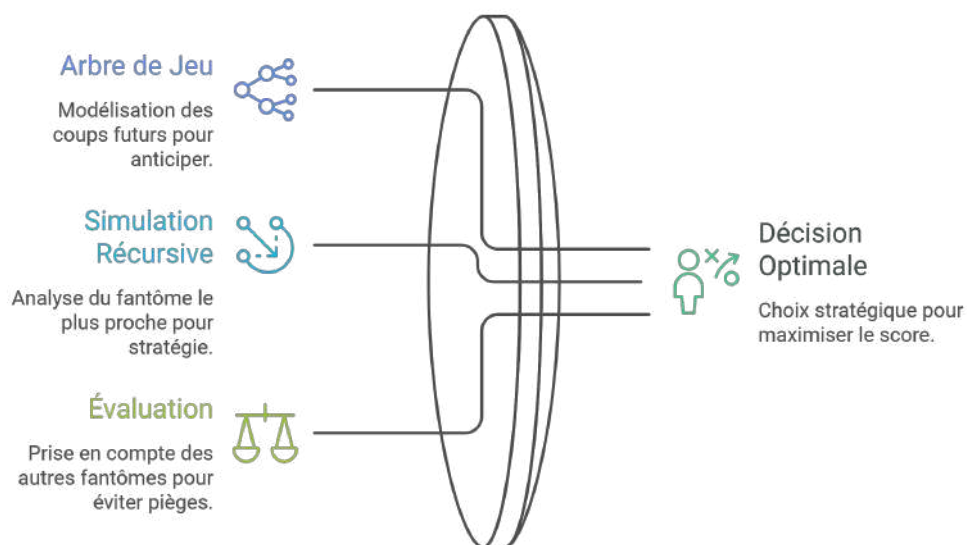


FIGURE 4 – Modélisation en arbre de jeu

6.2 Minimax

Dans Minimax, Pac-Man est le joueur MAX tandis que les fantômes représentent les joueurs MIN. Pac-Man cherche à maximiser son utilité alors que les fantômes cherchent à la minimiser, ce qui rend le comportement plus prudent et stratégique.

6.3 Alpha-Beta

Alpha-Beta optimise Minimax en supprimant les branches inutiles de l'arbre de recherche. Le résultat reste identique mais le nombre de calculs est fortement réduit.

6.4 Expectimax

Expectimax remplace les nœuds MIN par des nœuds probabilistes. Cette approche devient plus réaliste lorsque les fantômes possèdent un comportement aléatoire, notamment en mode Frightened.

6.5 Heuristiques et optimisations

Après l'implémentation de Minimax, Alpha-Beta et Expectimax, plusieurs comportements indésirables sont apparus comme des oscillations gauche-droite, des hésitations dans les couloirs ou encore des déplacements peu naturels.

Afin d'améliorer la qualité du comportement de l'IA, plusieurs heuristiques et optimisations pragmatiques ont été ajoutées.

La fonction d'évaluation combine notamment la distance aux fantômes dangereux, la recherche du pellet le plus proche ainsi qu'un système de progression récompensant les pellets consommés.

Des mécanismes anti-oscillation ont également été intégrés avec une mémoire des cellules récemment visitées et une pénalité lors des demi-tours immédiats.

Enfin, lorsqu'aucun fantôme menaçant n'est proche, le système utilise automatiquement un BFS rapide vers le pellet le plus proche afin de réduire le coût de calcul tout en conservant un comportement efficace.

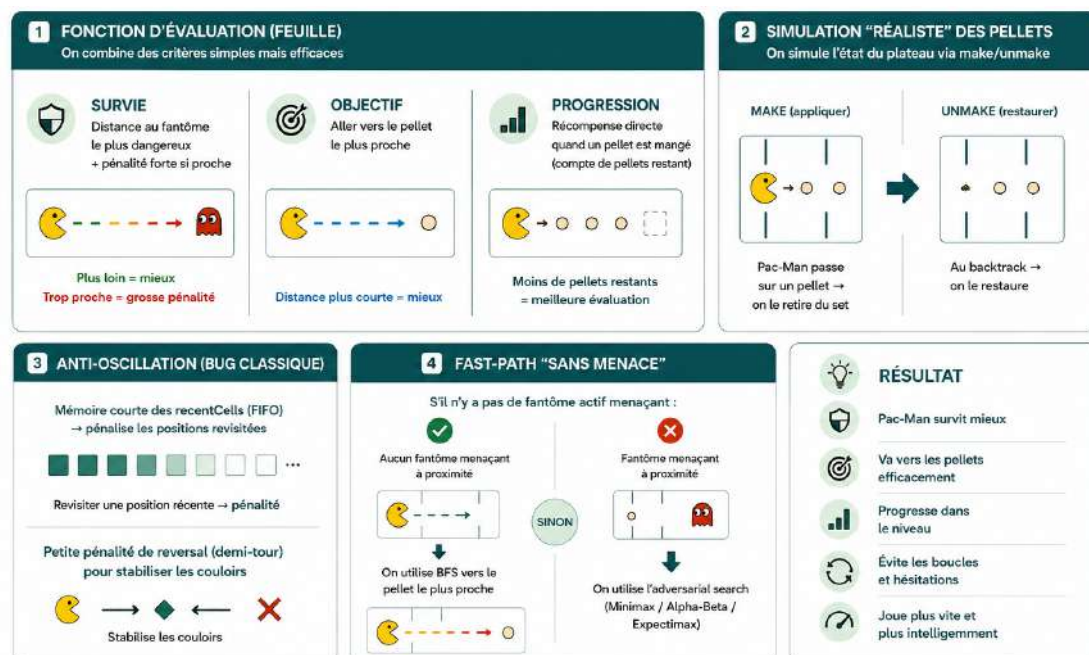


FIGURE 5 – Heuristiques et optimisations utilisées pour le Pac-Man Self-Play

7 Tests, statistiques et CI/CD

Afin de garantir la stabilité du projet, plusieurs systèmes de validation ont été mis en place.

7.1 Statistiques temps réel

Le système collecte dynamiquement plusieurs statistiques comme le nombre de nœuds explorés, le temps d'exécution, la longueur moyenne des chemins ou encore le taux de succès afin de comparer les algorithmes dans les mêmes conditions.

```
const AlgorithmStats = {  
  bfs: {  
    totalCalls: 0,           // Nombre d'appels  
    totalNodesExplored: 0,   // Nœuds visités  
    totalPathLength: 0,     // Longueur chemins  
    totalTimeMs: 0,         // Temps cumulé  
    pathsFound: 0,          // Succès  
    pathsNotFound: 0        // Échecs  
  },  
  // ... même structure pour astar et greedy  
};
```

FIGURE 6 – Système De Statistiques et Comparaison

7.2 Tests automatisés

Le projet possède 51 tests automatisés couvrant la génération de labyrinthes, les API, le gameplay, la qualité des labyrinthes ainsi que plusieurs scénarios end-to-end.

7.3 CI/CD

Une pipeline GitHub Actions exécute automatiquement les tests avant déploiement sur Render. Ainsi, un code instable ne peut pas atteindre la production.

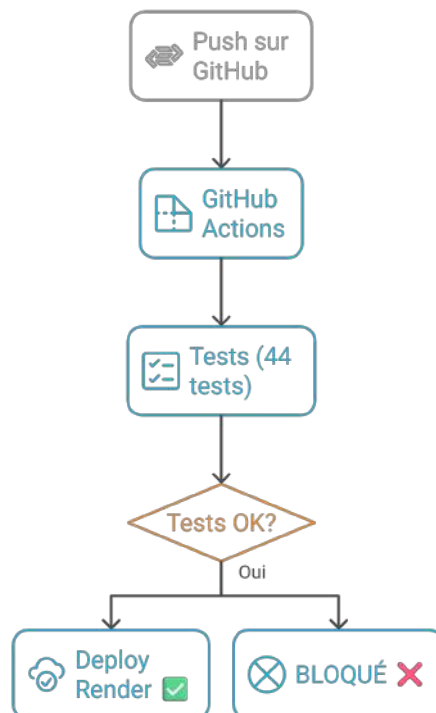


FIGURE 7 – Tests automatisés et pipeline CI/CD

8 Conclusion et perspectives

Ce projet nous a permis de combiner plusieurs domaines importants de l'informatique tels que la génération procédurale, l'intelligence artificielle, le développement web et l'optimisation temps réel.

Au cours du TER, nous avons développé un système complet autour de Pac-Man comprenant la génération automatique de labyrinthes, un moteur de jeu interactif, plusieurs algorithmes de pathfinding pour les fantômes, ainsi qu'une IA de type self-play basée sur Minimax, Alpha-Beta et Expectimax.

Le projet met particulièrement en avant la comparaison d'algorithmes en temps réel grâce à un système de statistiques, des tests automatisés et une pipeline CI/CD permettant de garantir la stabilité de l'application déployée sur le cloud.

Au-delà des résultats obtenus, ce projet ouvre également plusieurs perspectives d'amélioration comme l'intégration de techniques de Machine Learning, l'ajout d'un mode multijoueur, de leaderboards globaux, de heatmaps de performance ou encore de méthodes avancées de pathfinding hiérarchique et de coopération entre agents.

Ce projet nous a ainsi permis d'appliquer concrètement des concepts théoriques d'intelligence artificielle dans un environnement interactif, visuel et évolutif.

Merci pour votre attention